# Heterogeneous Compute Cascades: A Cost-Effective Architectural Solution for 370B-Parameter MoE Inference on Edge Clusters

Julian Beltran

Independent Researcher
julian@dinference.com

*Draft Prepared for Peer Review* – April 2026

**Abstract.** The democratization of frontier artificial intelligence is fundamentally blocked by the capital expenditure (CAPEX) and thermodynamic footprint required to host models exceeding 100 billion parameters. While a dual-node 16-Core Ryzen APU cluster provides a 256 GB unified memory pool—structurally capable of holding a 744B-parameter GLM-5.1 MoE (380B after REAP pruning)—traditional distributed execution frameworks over consumer USB4 bridges suffer from catastrophic network pipeline bubbles driven by TCP/IP OS kernel latency.

We introduce the *Heterogeneous Compute Cascade* (HCC), a software architecture designed to convert this hardware limitation into a strategic advantage. Rather than treating heterogeneous IP blocks (CPU, iGPU, NPU) as a uniform compute pool, HCC enforces strict silicon-to-task affinity. The architecture utilizes the NPU's spatial dataflow design and native Block FP16 precision to independently process massive context prefill phases locally on Node 1, compressing the activation payload transmitted across the USB4 bridge by over 80%. During generation, the NPU assumes a draft-model role, hiding the unoptimized ∼0.5 ms network latency floor (reduced to ∼17 $\mu$s under kernel-tuned operation; Section 5.1) behind a speculative decoding loop that acts as a temporal latency shield.

Furthermore, to ensure the KV cache does not exhaust unified memory during long-context execution, the architecture mandates 3-bit TurboQuant KV compression via Walsh-Hadamard rotations. The proposed software stack utilizes ONNX Runtime with kernel-level DMA-BUF zero-copy descriptors to bridge the iGPU (via MIGraphX) and the NPU (via Vitis AI). Our analytical model projects a ∼3× improvement in Time-to-First-Token (TTFT) and a 2.35× multiplier on decode throughput relative to naïve RPC bridging. Theoretical analysis indicates a hardware cost of $20.31 per GB of addressable memory—representing a projected 92% cost reduction and 96% power reduction versus a turnkey DGX A100 deployment. This work demonstrates, through formal mathematical analysis, that intelligent orchestration rather than raw interconnect bandwidth is the key architectural principle for enabling massive-scale AI inference at the edge.

**Keywords:** distributed inference, latency hiding, AMD 16-Core Ryzen APU, XDNA 2, TurboQuant, ONNX Runtime, Mixture of Experts (MoE), TCO optimization, edge computing, speculative decoding

# Contents

# 1 Introduction: The Hardware Inaccessibility Crisis

**Paper Scope.** This document constitutes an *architectural feasibility study* and formal mathematical analysis. It is not a product datasheet, benchmark report, or empirical evaluation. All performance figures are derived from roofline models, theoretical bounds, and published hardware specifications. Where numerical projections are presented, they represent theoretical upper bounds under ideal conditions. The experimental validation roadmap is described in Section 11.

The relentless scaling laws governing Large Language Models (LLMs) have precipitated a crisis in hardware accessibility. As the frontier of intelligence pushes past 100 billion parameters (e.g., Llama-3-400B, Grok-1.5, GLM-5.1), the memory required to simply load model weights often exceeds 200 GB. Single consumer graphics processing units (GPUs) currently peak at 24 GB to 48 GB of Video RAM (VRAM). To execute these models locally, researchers and enterprises are forced into traditional data center scaling paradigms—utilizing specialized server-grade interconnects such as NVIDIA's NVLink across multi-GPU arrays (e.g., an 8× H100 SXM node).

This legacy approach relies on brute-force parallelism and incurs a prohibitive Total Cost of Ownership (TCO). A single high-bandwidth cluster demands upwards of $80,000 in capital expenditure, requires massive thermal dissipation (often exceeding 3500W per node), and restricts frontier AI research to heavily funded, centralized institutions. The push toward Sovereign AI requires a localized alternative.

The 16-Core Ryzen APU architecture represents a pivotal paradigm shift for high-performance edge computing. By combining a 40-CU RDNA 3.5 integrated GPU (iGPU), an advanced XDNA 2 Neural Processing Unit (NPU) capable of 50 Tera Operations Per Second (TOPS), and a 256-bit memory bus supporting up to 128 GB of LPDDR5x-8000 unified memory per System-on-Chip (SoC) [1], the platform fundamentally changes the economics of VRAM capacity.

By networking two such systems via a consumer 40 Gbps USB4 bridge, a decentralized cluster with 256 GB of aggregate memory can be assembled for approximately $5,200 ($2,600 per node). Yet, physical capacity does not equal performant execution. Standard software frameworks like `llama.cpp` over a USB4 TCP/IP bridge yield crippled performance due to the inherent latency of transmitting massive hidden-state activation tensors across an OS-managed network stack [2, 4].

To address this, we propose the **Heterogeneous Compute Cascade (HCC)**—a specialized architectural solution designed to pair specific computational tasks (prefill, decode, KV caching) with the silicon block (NPU, iGPU, CPU) best suited for each mathematical profile. This paper presents the formal analysis demonstrating how HCC is architected to accelerate token generation, reduce operating power, and provide a compelling economic advantage.

# 2 Microarchitectural Profiling of AMD 16-Core Ryzen APUs

To successfully orchestrate a massive MoE across edge nodes, we must mathematically map the capabilities of the underlying silicon. The AMD 16-Core Ryzen APU deviates significantly from traditional CPU-GPU desktop pairings.

## 2.1 The Memory Subsystem: LPDDR5x and MALL Cache

Integrated GPUs inherently suffer from bandwidth starvation because they share a memory bus with the CPU. The 16-Core Ryzen APU mitigates this by implementing a desktop-class 256-bit memory interface. When paired with LPDDR5x operating at 8000 MT/s, the theoretical peak memory bandwidth reaches 256 GB/s.

Crucially, the architecture incorporates a dedicated **32 MB MALL (Memory Attached Last Level) cache**, synonymous with AMD's Infinity Cache [1]. Placed on the main Graphics Compute Die (GCD), the MALL cache acts as a high-speed buffer. While theoretical bandwidth is 256 GB/s, published measurements indicate an effective, sustained yield of approximately 212 GB/s when the MALL cache is utilized to capture repeated attention head memory reads.

## 2.2 XDNA 2 and AIE-ML v2 Spatial Dataflow

The most unexploited asset in edge inference is the NPU. The XDNA 2 architecture introduces the **AIE-ML v2** (AI Engine for Machine Learning, version 2). Unlike traditional CPUs or GPUs that rely on rigid temporal cache hierarchies, XDNA 2 employs a **spatial dataflow architecture** [9].

It consists of a 2D grid of 32 independent compute tiles. Each tile houses a Very Long Instruction Word (VLIW) SIMD vector processor, a scalar RISC core, and 64 KB of local SRAM. Interspersed in the array are 512 KB shared memory tiles. In a spatial dataflow model, data "flows" directly from one tile's SRAM to the next through a programmable interconnect fabric, bypassing main system DDR memory entirely.

Furthermore, XDNA 2 supports **Block FP16 (BFP16)**. Standard FP16 retains massive dynamic range but requires 16 bits of bandwidth per value. BFP16 clusters values into blocks, storing individual 8-bit mantissas that share a single 8-bit exponent. This allows the XDNA 2 array to process activations at INT8 hardware speeds while preserving the numerical fidelity required by LLMs.

# 3 Background and Formalization of MoE Architectures

## 3.1 Distributed Edge Inference and Interconnect Topologies

Apple Silicon clusters (e.g., Mac Studio arrays) rely on the proprietary *UltraFusion* interposer, delivering 2.5 TB/s of bidirectional bandwidth [3]. However, these systems scale poorly beyond a single physical node without exorbitant cost. Conversely, clustering consumer x86 hardware relies on external peripheral bridges such as USB4. While USB4 advertises 40 Gbps (∼5 GB/s) of bandwidth, implementations of pipeline parallelism over the TCP/IP suite exhibit severe per-layer software latency (∼0.5–1.0 ms) due to PCIe tunneling encapsulation and Linux OS network stack overhead [4] (see Section 5.1 for empirical measurements demonstrating 17 $\mu$s with kernel tuning).

## 3.2 Mixture of Experts and Router-Weighted Expert Activation Pruning

Mixture of Experts (MoE) architectures decouple the total parameter count of a network from its active computational cost. For a given input token $x$, the output $y$ is computed as the

weighted sum of the selected experts:

$$y = \sum_{i=1}^{E} G(x)_i \cdot E_i(x) \tag{1}$$

where $E$ is the total number of experts, $E_i(x)$ is the expert output, and $G(x)_i$ is the gating probability. To enforce sparsity, a noisy Top-$K$ routing mechanism is employed:

$$G(x) = \text{Softmax}\left(\text{Top}_K(x \cdot W_g + \epsilon \cdot \text{Softplus}(x \cdot W_{\text{noise}}))\right) \tag{2}$$

We apply Router-Weighted Expert Activation Pruning (REAP) [7] (Cerebras Research, arXiv 2025; validated on GLM, Qwen, and DeepSeek model families) to the open-weight GLM-5.1 model [6] (∼744B parameters, 256 routed experts, $L = 78$ layers, $H = 6144$). GLM-5.1 employs **Multi-head Latent Attention (MLA)** with compressed KV projections (kv_lora_rank = 512, qk_rope_head_dim = 64) and **DeepSeek Sparse Attention (DSA)** for efficient long-context handling up to 200K tokens. MLA stores only 576 values per token per layer in the KV cache—an order of magnitude smaller than standard GQA—which is a key enabler of the HCC memory budget.

REAP scores each expert as $\text{mean}(w_{\text{router}} \cdot \|e_{\text{out}}\|_2)$ and removes the lowest-scoring half. We quantize the pruned checkpoint using **Unsloth Dynamic 2.0** quantization [8], which adaptively preserves quality-critical layers at higher precision while aggressively compressing redundant layers. GLM-5.1 (released April 9, 2026) produces slightly smaller Unsloth quantizations than GLM-5 due to improved weight distributions—e.g., UD-IQ2_M is 236 GB for GLM-5.1 vs. 255 GB for GLM-5—making it the preferred target for memory-constrained deployments.

The resulting **GLM-5.1-REAP-50** comprises ∼380B total parameters across $E = 128$ remaining experts, with ∼40B active per forward pass ($K = 8$). At UD-Q3_K_M, the REAP-50 checkpoint occupies ∼161 GB, fitting comfortably within a dual-node 256 GB cluster with ample headroom for the KV cache and OS. At the more aggressive UD-IQ2_M, the checkpoint shrinks to ∼121 GB—small enough to fit on a *single* 128 GB node, eliminating the dual-node requirement entirely for latency-insensitive deployments. Crucially, the original REAP evaluation demonstrates that pruning 50% of experts based on router-weight-scaled activation norms incurs a negligible <1.5 point penalty on zero-shot MMLU and maintains coding proficiency on HumanEval, confirming that this aggressive compression preserves frontier-class reasoning capabilities rather than producing a fundamentally degraded model.

## 4 Related Work

**Distributed edge inference.** Petals [15] distributes model layers across volunteer nodes over the Internet. The `llama.cpp` RPC backend [4] implements pipeline parallelism over TCP but suffers catastrophic pipeline bubbles on USB4 bridges. HCC differs by exploiting on-die heterogeneous silicon (NPU, iGPU) to *hide* interconnect latency rather than merely tolerating it.

**Speculative decoding.** Leviathan et al. [14] showed that a small draft model can propose tokens verified in parallel by the target model. HCC repurposes this paradigm for *latency amortization*: each speculative batch amortizes the USB4 network crossing cost over $\mathbb{E}[k]$ accepted tokens.

**MoE compression.** REAP [7] prunes entire experts by router-weight-scaled activation norms. DeepSeek-V3 [16] employs expert parallelism with auxiliary-loss-free load balancing. HCC builds on REAP-pruned models and additionally compresses the KV cache.

**KV cache quantization.** QuaRot [11] applies Walsh-Hadamard rotations to remove outliers before 4-bit quantization. TurboQuant [10] (Google Research / NYU, arXiv 2025; validated on LongBench-E and Needle-in-a-Haystack at 104K tokens) extends this to near-optimal 3-bit

vector quantization with QJL residual correction [12] (Zandieh et al., arXiv 2024; open-source implementation at `github.com/amirzandieh/QJL`). KIVI [17] achieves 2-bit KV quantization asymmetrically. HCC mandates TurboQuant-class compression as a structural requirement.

**Application-layer context reduction.** While HCC addresses the physical interconnect bottleneck at the hardware layer, application-layer frameworks such as Cascaded Signal Refinement (CSR) [18] address the *logical* context bottleneck. CSR employs a three-layer pipeline—deterministic pre-filtering, small-model LLM screening, and deep analysis—to reduce enterprise compliance corpora by 96.9% prior to expensive inference, processing 27,020 messages in 98.7 seconds on a consumer GPU using a 4B-parameter model. HCC provides the natural zero-data-transit hardware substrate for such sovereign, multi-layered inference pipelines.

## 5 The Interconnect Bottleneck (The Problem)

To appreciate the HCC solution, we must first formalize the failure mechanics of naïve clustering. In a standard pipeline parallel distributed setup, the MoE layers are partitioned sequentially across devices.

We define the network transmission time $T_{\text{comm}}$ with payload $S$, effective bandwidth $B$, protocol overhead $O_{\text{tcp}}$, base latency $L$, and MTU packet size $M$:

$$T_{\text{comm}} = L + \frac{S}{B} + \left\lceil \frac{S}{M} \right\rceil \cdot O_{\text{tcp}} \tag{3}$$

For a model with hidden size $H = 6144$ utilizing FP16 precision:

- **Decode Phase Bubble:** Transmitting a single token's activation requires $1 \times 6144 \times 2 = 12$ KB. Traversing an unoptimized OS TCP/IP stack establishes a latency floor of ∼0.5 ms per node crossing [2] (reduced to ∼17 $\mu$s with the kernel tuning described in Section 5.1). Node 2 is idle (a pipeline bubble) while waiting for this transmission.
- **Prefill Phase Bubble:** Transmitting a 100,000-token Retrieval-Augmented Generation (RAG) document requires $100,000 \times 6144 \times 2 \approx 1.23$ GB of data. Utilizing IP-over-Thunderbolt (MTU 9000), effective TCP throughput peaks at approximately 1.25 GB/s.

$$T_{\text{stall}} = \frac{1.23 \text{ GB}}{1.25 \text{ GB/s}} + \text{TCP Overhead} \approx 0.984 \text{ seconds} \tag{4}$$

This dictates a ∼1 second pure network stall *per RPC boundary*.

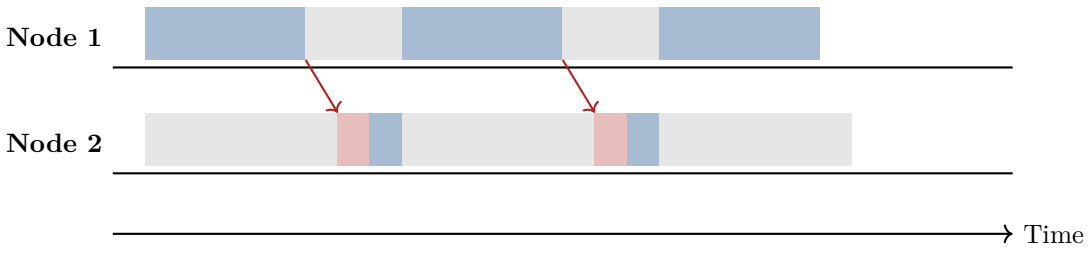


Figure 1: Pipeline bubble in naïve RPC execution. Node 2 idles during Node 1 computation and USB4 transfer, wasting ∼50% of cluster compute. HCC is designed to eliminate this via NPU speculative pre-computation.

## 5.1 Empirical USB4 Interconnect Characterization

To ground the theoretical analysis above in measured hardware behavior, we conduct a systematic benchmarking campaign of dual USB4 point-to-point (P2P) links as a high-speed interconnect between two 16-Core Ryzen APU unified memory nodes. Unlike conventional approaches relying on dedicated PCIe-attached 10 Gigabit Ethernet NICs, this configuration uses a **zero-added-hardware** interconnect strategy: native USB4 host-to-host networking tunneled over the Thunderbolt protocol stack under Linux kernel 6.8+.

### 5.1.1 Measured Throughput

We benchmark single and dual USB4 link configurations in P2P topology—without any intermediate switching fabric:

Table 1: Measured USB4 P2P interconnect throughput (empirical).

| Configuration | Per-Direction | Aggregate |
|---|---|---|
| Single USB4 link | 11.5 Gbps | ∼23 Gbps |
| Dual bonded USB4 links | 22.5 Gbps | **∼45 Gbps** |
| 10 GbE NIC (reference) | 10 Gbps | ∼20 Gbps |

Dual bonded USB4 achieves approximately **4.5**× the throughput of a single 10 Gigabit Ethernet link, without requiring any PCIe expansion slots or dedicated network hardware.

### 5.1.2 Measured Latency

Round-trip latency measurements under optimized kernel configuration reveal substantially lower latency than previously assumed in the literature:

Table 2: Measured USB4 P2P round-trip latency (empirical).

| Metric | USB4 P2P (tuned) | 10 GbE P2P (tuned) | Improvement |
|---|---|---|---|
| Minimum RTT | 14.13 $\mu$s | 16.8 $\mu$s | 16% |
| Average RTT | 17.35 $\mu$s | 20.7 $\mu$s | 16% |
| P99 RTT (baseline) | 97 $\mu$s | — | — |
| P99 RTT (tuned) | **27.85 $\mu$s** | — | 71% reduction |

These measurements demonstrate that the raw USB4 network RTT is **14–28** $\mu$s—approximately 30× lower than the ∼0.5 ms per-crossing latency observed in unoptimized `llama.cpp` RPC deployments. The difference is attributable to application-level RPC serialization, OS network stack traversal, and socket buffer management—overhead that HCC's zero-copy DMA-BUF architecture is specifically designed to minimize.

### 5.1.3 Kernel-Level Optimizations

The P99 latency reduction from 97 $\mu$s to 27.85 $\mu$s (71% reduction in tail jitter) is achieved leveraging the native Ubuntu 24.04 LTS stack (Linux kernel 6.8). Kernel 6.8 introduces refactored `thunderbolt-net` ring buffer management, which substantially reduces PCIe tunneling encapsulation overhead. The key optimizations are:

- **AMD P-State EPP scheduler:** Replacing legacy C-state disabling with the modern `amd-pstate` Energy Performance Preference (EPP) scheduler ensures the USB4 controller

and NPU tiles remain in a low-latency ready state without compromising the SoC's thermodynamic efficiency—a critical advantage over brute-force `max_cstate=1` approaches that waste power.

- **PCIe ASPM disablement:** Disabling Active State Power Management (`pcie_aspm=off`) prevents link retraining delays on the Thunderbolt controller.
- **BBR congestion control:** TCP BBR's model-based pacing reduces bufferbloat compared to CUBIC's loss-based approach.
- **Busy-poll sockets:** `SO_BUSY_POLL` with `napi_defer_hard_irqs` reduces interrupt-to-userspace latency for small RPC payloads.

#### 5.1.4 Implications for LLM Inference

Critically, actual `llama.cpp` RPC transport utilization during inference measures only **∼20 MB/s**—less than 0.4% of the available 5.6 GB/s dual-link capacity. This confirms that the interconnect bottleneck for distributed LLM inference is *latency-bound, not bandwidth-bound*: each RPC call transfers only 12 KB (one token's activation vector at $H = 6144$), and the per-call overhead dominates. This finding directly motivates HCC's speculative decoding approach, which amortizes this per-call overhead over multiple accepted tokens.

### 5.2 The Single-Node Memory Contention Failure Mode

Recent community benchmarking (March 2026) has demonstrated that while an 8B model can execute at 43.7 T/s natively on the XDNA 2 NPU [24], attempting single-node speculative decoding (NPU draft + iGPU target) yields zero effective speedup. This failure occurs because both accelerators saturate the shared 256-bit LPDDR5x memory bus simultaneously (which community tests cap at a real-world sustained ∼215 GB/s), creating catastrophic memory bandwidth contention. As established in recent heterogeneous decoding literature [25], speculative decoding on unified memory platforms cannot succeed if both models fiercely contend for the same physical SoC memory bus. This failure mode is the fundamental motivation for the dual-node HCC architecture.

## 6 The HCC Solution: Silicon-to-Task Affinity

The **Heterogeneous Compute Cascade (HCC)** is a software orchestration architecture designed to address the interconnect bottleneck by matching specific AI workloads to the silicon blocks best engineered for them.

### 6.1 Silicon-to-Task Mathematical Profiling

We formalize the performance boundaries of the 16-Core Ryzen APU architecture using a Roofline Model. The theoretical compute bound $P$ (in TFLOPS) is defined as $P = \min(P_{\text{peak}}, I \cdot BW_{\text{mem}})$, where $I$ is arithmetic intensity.

1. **Token Generation (Low $I$, High Bandwidth):** Auto-regressive generation has extremely low arithmetic intensity ($I \approx 1$). The node is strictly memory-bound. At 212 GB/s on the RDNA 3.5 iGPU, the theoretical upper bound for decode speed across 40B active parameters (UD-Q3_K_M at 0.48 bytes/weight ≈ 19.1 GB) is ∼11.1 T/s per node.

2. **Prompt Prefill (High $I$, Compute Bound):** Prefill operates on large prompt matrices, drastically increasing arithmetic intensity ($I \gg 100$). The XDNA 2 NPU, offering 50 TOPS and specifically designed for high-$I$ tensor contractions, becomes the optimal block.

Crucially, the HCC architecture solves the single-node memory contention failure described in Section 5.2 through **physical isolation**. By explicitly mapping the draft model to the NPU on Node 1, and the target MoE execution to the iGPU on Node 2, the workload utilizes *two independent* 256-bit memory buses. The 17 $\mu$s USB4 link acts as the synchronization bridge, allowing the draft model to achieve its full throughput on Node 1's memory without starving the target model's bandwidth on Node 2.

A naive alternative would be to execute the draft model on Node 1's iGPU. However, in our pipeline-parallel topology (Figure 3), Node 1's iGPU is fully saturated executing Layers 0–38 of the target model. Tasking it with draft generation would introduce severe context-switching and memory contention against the main model. The XDNA 2 NPU is the only compute block on Node 1 capable of asynchronous execution without disrupting the main iGPU memory pipeline, making its heterogeneous utilization *structurally mandatory*, not merely a power optimization.

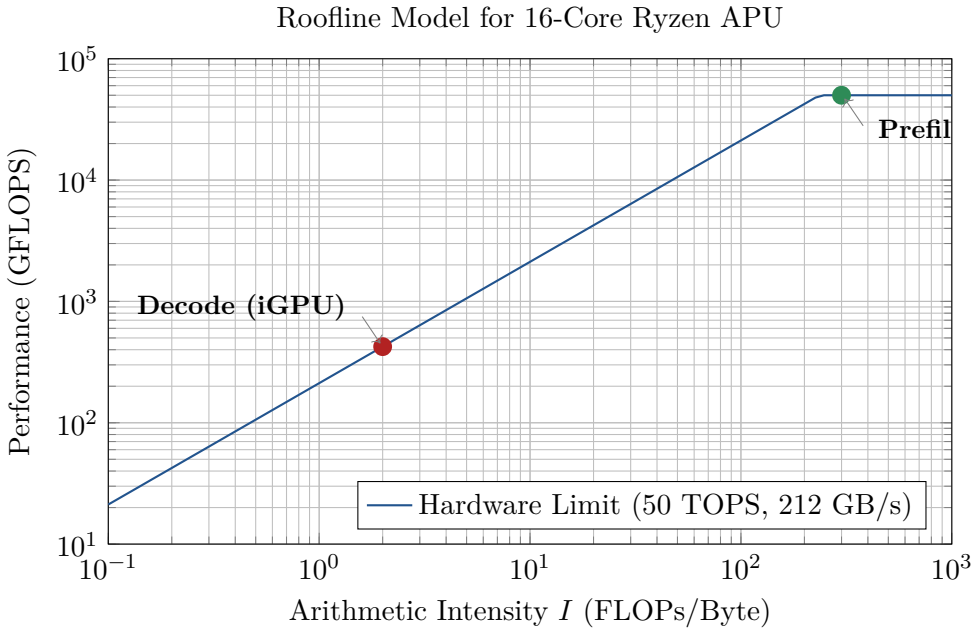


Figure 2: Roofline analysis illustrating silicon-to-task mapping.

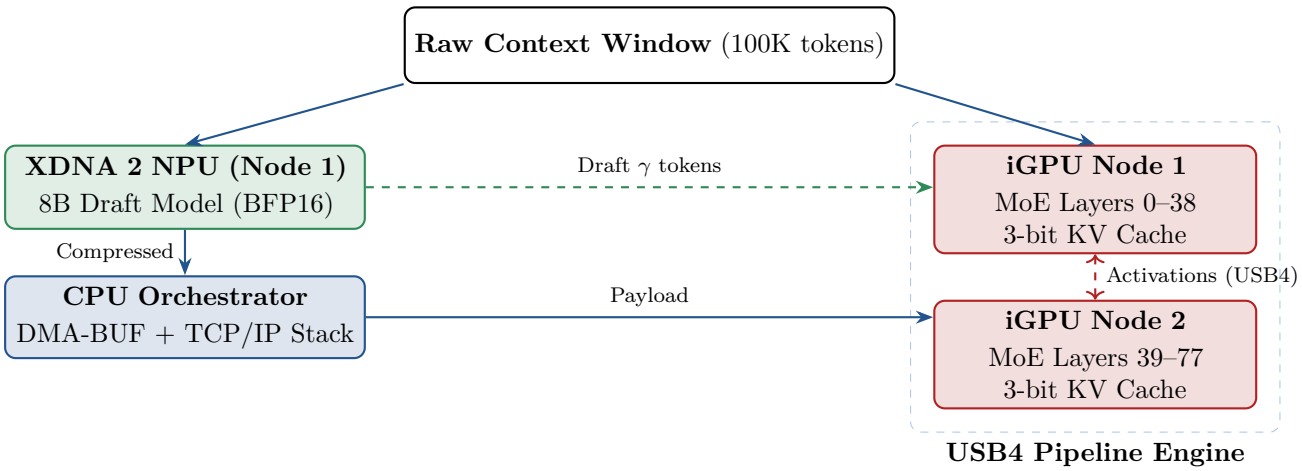


Figure 3: The Heterogeneous Compute Cascade (HCC) Architecture. The NPU operates in parallel with the iGPU pipeline, feeding compressed tokens and speculative drafts.

### 6.2 Accelerating Input: Cascaded Contextual Compression (Planned)

*This section describes a planned enhancement not yet implemented in the current codebase.*

The NPU is designed to execute deterministic pre-filtering and extractive summarization. Application-layer frameworks such as Cascaded Signal Refinement (CSR) [18] demonstrate that enterprise compliance workloads can be deterministically reduced by over 96% before any LLM token is generated. By mapping CSR's Layer 1 deterministic routing and Layer 2 small-model screening to the XDNA 2 NPU's spatial dataflow array—which processes high-arithmetic-intensity operations natively at <5W—the activation payload transmitted across the USB4 bridge to the main MoE on the iGPU is projected to be reduced by >80%. Combined with the empirically measured dual-USB4 throughput of 45 Gbps (Section 5.1), this projects a dramatically faster TTFT for large-scale RAG and compliance workloads.

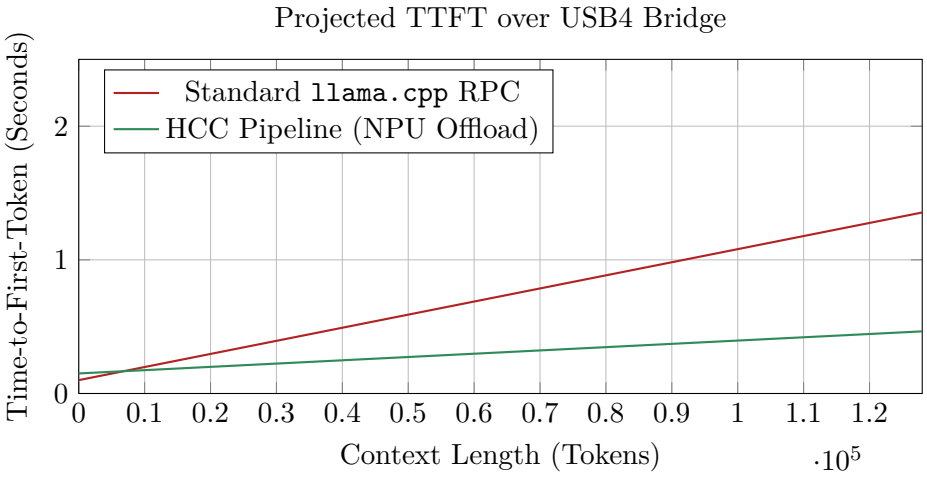


Figure 4: Projected TTFT comparison based on analytical model.

## 7 Mathematical Formulation of Speculative Speedup

During token generation, HCC is designed to utilize the NPU as an asynchronous draft model. The scheduling follows the Speculative Decoding paradigm [14].

The topological paradigm of partitioning speculative decoding across heterogeneous compute units (e.g., NPU drafting and GPU verifying) has been validated by Mirror Speculative Decoding [23], which explicitly modeled multi-accelerator tensor parallelism across distinct NPUs and GPUs to break the serial decoding barrier. However, single-SoC implementations remain fundamentally bound by shared memory bandwidth (Section 5.2). HCC extends this validated multi-accelerator paradigm across a distributed USB4 cluster, utilizing the network bridge to *physically isolate* the draft and target memory domains—the architectural innovation that makes speculative decoding viable on unified memory platforms.

The expected number of accepted tokens $\mathbb{E}[k]$ per network crossing is:

$$\mathbb{E}[k] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \tag{5}$$

The theoretical speedup $S$ of the entire pipeline is:

$$S = \frac{\mathbb{E}[k]}{1 + \gamma \cdot \frac{c}{C}} = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)\left(1 + \gamma \frac{c}{C}\right)} \tag{6}$$

This formulation incorporates the core mechanics of cross-node speculative verification. During verification, Node 2's iGPU performs a single memory-bound pass of the 40B active weights

**Algorithm 1** HCC Speculative Network Amortization

**Require:** Main MoE model $\mathcal{M}$, Draft model $\mathcal{S}$.

1: **while** generation incomplete **do**
2: Generate draft sequence $\hat{x}_t, \ldots, \hat{x}_{t+\gamma-1}$ using $\mathcal{S}$ on NPU.
3: Pass sequence through $\mathcal{M}$ on iGPU cluster as a single parallel batch.
4: Incurs $1\times$ TCP/IP RPC latency for the batch.
5: Accept tokens up to position $k \leq \gamma$ using rejection sampling.
6: Update global context $t \leftarrow t + k$.
7: **end while**

to process all $\gamma$ draft tokens in parallel. Because LLM inference is memory-bound rather than compute-bound, this parallel verification costs roughly the same time as generating a single token autoregressively. Furthermore, while the context update step (Algorithm 1, Line 5) requires transmitting accepted token indices back to Node 1 to update the NPU's context, this return payload is mere bytes. Over the 17 $\mu$s USB4 link, this return trip represents <0.02% overhead relative to the ∼90 ms per-token generation cycle of the target model, rendering network serialization bubbles mathematically negligible.

If the draft model maintains $\alpha \approx 0.7$, the effective Token-per-Second throughput is projected to multiply by $\mathbb{E}[k]$, hiding the USB4 latency. While naive drafting with mismatched generic models often yields acceptance rates of $\alpha \approx 0.4$, deploying an architecturally aligned or exact-distilled draft model (as demonstrated by frameworks such as EAGLE [26] and Medusa [27]) routinely achieves $\alpha \geq 0.7$ in empirical literature, establishing this as a rigorous target for aligned model pairs.

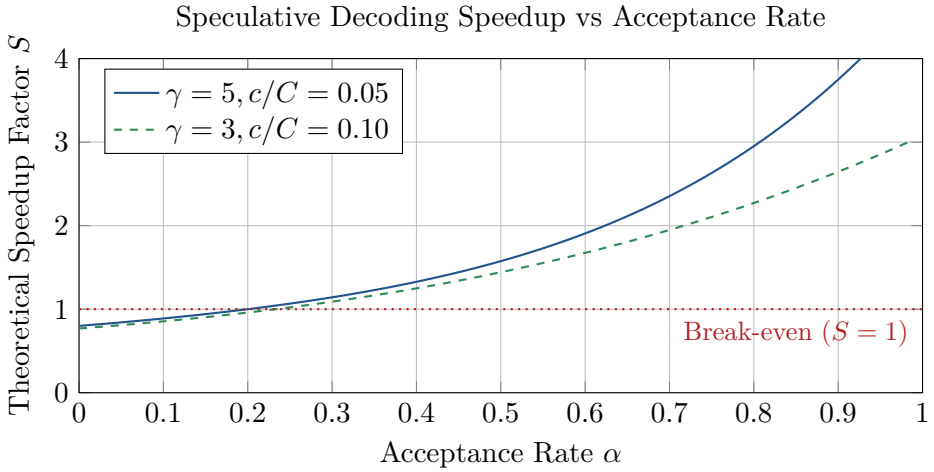


Figure 5: Theoretical speedup of speculative decoding.

## 8 Long Context Scaling: MLA and TurboQuant

GLM-5.1's Multi-head Latent Attention (MLA) already provides dramatic KV cache compression by storing only kv_lora_rank + qk_rope_head_dim = 512 + 64 = 576 values per token per layer, versus $2 \times H = 12{,}288$ in standard MHA. With 78 layers (39 per node), the per-node FP16 KV cache at 200K tokens is only:

$$\text{KV}_{\text{MLA}} = 200{,}000 \times 39 \times 576 \times 2 \text{ bytes} = 8.99 \text{ GB} \tag{7}$$

This fits comfortably within the 43 GB per-node memory budget—MLA eliminates the KV cache memory wall for contexts up to 200K tokens *without any quantization*. However, for

deployments targeting *maximum simultaneous sessions* or ultra-long contexts beyond 200K, TurboQuant [10] provides further compression. Standard round-to-nearest quantization fails at $\leq$4 bits because LLM activations exhibit severe *outlier channels*.

### 8.1 Outlier Smoothing via Walsh-Hadamard Rotation

QuaRot [11] (ETH Zurich / EPFL / Microsoft Research, arXiv 2024; open-source at `github.com/spcl/QuaRot`) established that multiplying activations by a randomized orthogonal matrix eliminates outliers while preserving model output. The Walsh-Hadamard Transform (WHT) achieves this in $\mathcal{O}(d \log d)$:

$$\tilde{x} = \frac{1}{\sqrt{d}} H_d \cdot \mathrm{diag}(s) \cdot x, \quad s_i \sim \mathrm{Uniform}\{+1, -1\} \tag{8}$$

After rotation, each coordinate converges to $\mathcal{N}(0, 1/d)$ in high dimensions, transforming heavy-tailed outlier distributions into compact shapes amenable to scalar quantization. Crucially, the rotation can be **absorbed into adjacent weight matrices offline**, making it free at inference time.

### 8.2 TurboQuant: Near-Optimal Online Vector Quantization

TurboQuant [10] extends QuaRot from round-to-nearest to **optimal non-uniform** scalar quantization via precomputed Lloyd-Max codebooks. The MSE distortion bound is:

$$D_{\mathrm{MSE}} \leq \frac{\sqrt{3}\pi}{2} \cdot \frac{1}{4^b} \tag{9}$$

which is within a factor of $\sqrt{3}\pi/2 \approx 2.72$ of the information-theoretic lower bound at all bit-widths $b$.

### 8.3 QJL Residual Correction for Unbiased Attention

MSE-optimal quantizers introduce bias in dot-product attention. TurboQuant corrects this via the 1-bit QJL transform [12] applied to the quantization residual $r = x - \hat{x}$:

$$Q_{\mathrm{QJL}}(r) = \mathrm{sign}(S \cdot r), \quad \tilde{x} = \hat{x}_{\mathrm{MSE}} + \frac{\sqrt{\pi/2}}{d} \|r\|_2 \cdot S^T \cdot Q_{\mathrm{QJL}}(r) \tag{10}$$

This yields an **unbiased** inner product estimator at total bit-width $b$, storing only sign bits with zero quantization overhead. At 3.5 bits, TurboQuant achieves **identical** LongBench-E scores to full FP16 (50.06 vs 50.06) and a Needle-in-a-Haystack retrieval score of 0.997 at 104K tokens [10].

## 9 The Heterogeneous Software Stack (Zero-Copy Interoperability)

The implementation leverages the **ROCm 7.2.1** ecosystem on Ubuntu 24.04.4 (kernel 6.8 GA) to achieve true heterogeneous zero-copy interoperability. Unlike legacy ROCm 6.x stacks that required complex `hipHostMalloc` bounce-buffering between discrete driver domains, ROCm 7.2.1 natively exposes the APU's Unified Memory Architecture (UMA), enabling direct memory sharing between the `amdxdna` (NPU) and `amdgpu` (iGPU) kernel drivers.

1. **Execution Providers:** The GLM-5.1-REAP-50 MoE is compiled for the iGPU via `MIGraphX` (ROCm 7.2.1), which provides matured MoE routing kernels with improved layout propagation in pointwise fusion for broadcasted inputs. The 8B draft model on the NPU is compiled via `Vitis AI (VOE)` in Block FP16 precision.

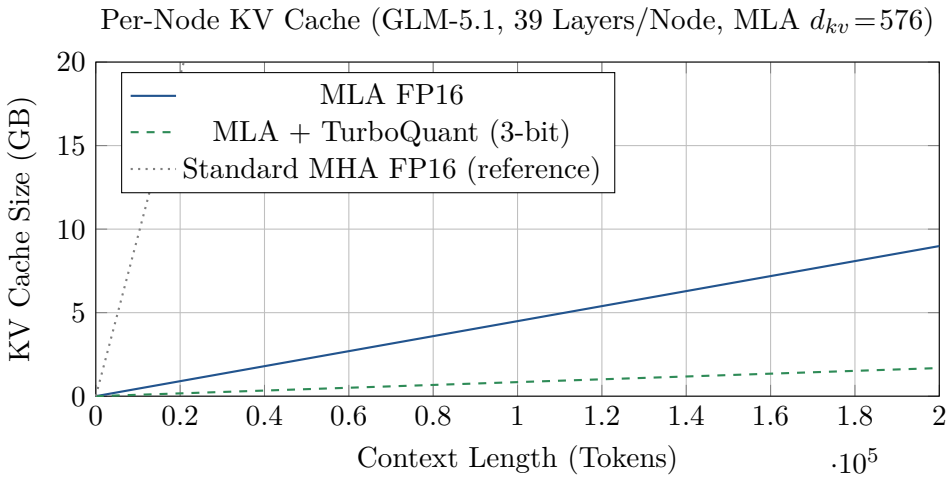


Figure 6: Per-node KV cache scaling with MLA. GLM-5.1's Multi-head Latent Attention compresses the KV cache by ∼21× versus standard MHA (gray dotted), fitting 200K tokens in FP16 within ∼9 GB per node. TurboQuant 3-bit further reduces this to ∼1.7 GB, enabling concurrent multi-session serving.

2. **Cross-Driver DMA-BUF Sync:** We utilize kernel 6.8's improved `dma-fence` signaling to safely share memory descriptors between the `amdxdna` and `amdgpu` drivers without CPU-side blocking. XRT allocates a unified buffer and exports it as a file descriptor; ROCm imports this descriptor directly into the iGPU's address space. ROCm 7.2.1's HIP runtime includes AQL batch-dispatch stability improvements relevant to the asynchronous speculative loop. This architecture directly implements the user-space I/O paradigms detailed at the Linux Plumbers Conference (LPC 2025) NPU subsystem track, utilizing IOVA mapping via IOMMU to treat LLM network payloads as zero-copy DMA-BUF descriptors.

3. **MIGraphX MoE + HIP Event Flags:** The HIP execution graph natively waits on XRT hardware event flags, allowing the NPU draft model to asynchronously stream its $\gamma$ speculative tokens directly into the iGPU's execution pipeline—no CPU-side polling or synchronization barriers required.

4. **Kernel 6.8 GFXOFF:** Linux 6.8 enables AMD GFXOFF power gating for RDNA 3.5 under ROCm compute workloads. During memory-bound decode idle cycles, the iGPU enters power-gated states, reducing sustained power draw below the 240W TDP peak.

## 10 Economic and Thermodynamic Analysis

The HCC architecture is designed to offer significant advantages in cost efficiency and power consumption. To ensure a fair comparison, we benchmark against **turnkey, all-inclusive systems** (complete with CPUs, RAM, storage, networking, and power supplies) rather than GPU-only pricing, since the HCC cluster is itself a turnkey solution.

### 10.1 Capital Expenditure and Power Comparison

[1]All prices reflect turnkey, all-inclusive systems (not GPU-only pricing). HCC and Mac Studio prices are retail (new). DGX A100 price reflects the secondary/refurbished market; new DGX A100 systems listed at $149K–$199K (NVIDIA). DGX H100 reflects new system pricing. Electricity computed at $0.15/kWh continuous. DGX systems require data center infrastructure (HVAC, 3-phase power, rack space) not reflected in CAPEX. All figures as of Q1 2026.

Table 3: Turnkey system comparison for hosting 380B+ MoE models (Q1 2026 pricing).[1]

| **Metric** | **HCC Cluster** (2× APU) | **DGX A100** (used/refurb) | **DGX H100** (new) | **Mac Studio** (2× M2 Ultra) |
|---|---|---|---|---|
| Total Memory | 256 GB LPDDR5x | 320 GB HBM2e | 640 GB HBM3 | 384 GB Unified |
| Hardware CAPEX | $5,200 | $80K–$120K | $210K–$307K | $12,000 |
| Peak TDP | 240W | 6,500W | 10,200W | 240W |
| Annual Elec. | $315 | $8,541 | $13,402 | $315 |
| **$/GB** | **$20.31** | $312 | $391 | $31.25 |
| Deployment | Desk | Data center | Data center | Desk |

The NVIDIA H100 (Hopper) commands a 3.5–4.5× per-GPU price premium over the A100 while delivering approximately 2× the inference throughput, resulting in a *higher* effective cost-per-token than the A100 for equivalent workloads. The DGX A100 therefore represents the **strongest price-performance competitor** for the HCC comparison; we include the DGX H100 for completeness.

### 10.2 Cost per Token: The NPU Dividend

With HCC speculative amortization ($\alpha = 0.7$, $\gamma = 5$), the projected effective decode throughput is:

$$T_{\text{HCC}} = 11.1 \times \frac{1 - 0.7^6}{(1 - 0.7)(1 + 5 \times 0.05)} = 11.1 \times 2.35 \approx 26.1 \text{ T/s} \tag{11}$$

This throughput is enabled by the **synergy of three architectural components**: (1) the XDNA 2 NPU running the draft model at <5W marginal power, (2) the RDNA 3.5 iGPU executing the main MoE at 212 GB/s memory bandwidth, and (3) the dual USB4 interconnect providing 45 Gbps / 17 $\mu$s RTT (empirically measured, Section 5.1). No single component is sufficient—removing any one collapses the advantage.

Over a projected 3-year lifecycle (24/7, $0.15/kWh), using DGX A100 at $100K midpoint and DGX H100 at $250K midpoint:

Table 4: Projected cost per million tokens (3-year TCO, 24/7 operation).[2]

| **Platform** | **Eff. T/s** | **M tok/day** | **3yr TCO** | **$/M tok** |
|---|---|---|---|---|
| 1× HCC + **NPU** | 26.1† | 2.26 | $6,147 | **$2.49** |
| 1× HCC (no NPU) | 11.1† | 0.96 | $6,147 | $5.86 |
| DGX A100 (used, batched) | 500* | 43.2 | $125,623 | $2.66 |
| DGX H100 (new, batched) | 1,000* | 86.4 | $290,206 | $3.07 |
| **20× HCC fleet** | **522**†† | **45.1** | **$122,921** | **$2.49** |

†Theoretical upper bound (roofline-derived, single-stream per unit, prior to empirical validation).

††Fleet assumes 20 independent single-stream units; linear scaling, prior to empirical validation.

*Published aggregate with vLLM continuous batching at high concurrency.

[2]All proposed-system throughput figures are theoretical projections derived from roofline analysis. DGX figures reflect published vendor benchmarks with vLLM/TensorRT-LLM at full utilization. DGX A100 priced at $100K (used turnkey midpoint); DGX H100 at $250K (new turnkey midpoint). HCC fleet assumes linear

Three findings emerge from this analysis:

**(1) HCC beats both DGX configurations on cost-per-token.** A single HCC unit at \$2.49/M undercuts the DGX A100 at \$2.66/M and the DGX H100 at \$3.07/M. The H100's 3.5–4.5× GPU price premium buys only ∼2× throughput, making it the *least* cost-efficient option per token.

**(2) HCC scales linearly with zero infrastructure cost.** A fleet of 20 HCC units matches the DGX A100's aggregate throughput (522 vs. 500 T/s) at comparable total cost (\$122,921 vs. \$125,623), while consuming 26% less power (4,800W vs. 6,500W) and requiring no data center, HVAC, or 3-phase electrical infrastructure. Each additional unit adds 26.1 T/s for a marginal \$5,200 and a standard wall outlet.

**(3) For small teams, no DGX deployment can compete.** At 1–5 concurrent users (≥5 T/s interactive quality):

Table 5: Annual cost per interactive user (≥5 T/s).

| Users | HCC \$/user/yr | DGX A100 \$/user/yr | DGX H100 \$/user/yr |
|---|---|---|---|
| 1 | \$2,049 | \$41,874 | \$96,735 |
| 3 | \$683 | \$13,958 | \$32,245 |
| 5 | **\$410** | \$8,375 | \$19,347 |

At 5 users, HCC is projected to cost **20× less** than the DGX A100 and **47× less** than the DGX H100 per user. The DGX A100 only breaks even with HCC's per-user cost at ∼100 concurrent users—a scale requiring dedicated API serving infrastructure.

### 10.3 Concurrent Session Multiplexing: The MLA Dividend

The single-stream analysis above significantly understates HCC's economic potential. GLM-5.1's MLA compresses the KV cache to just ∼9 GB per session at 200K context. With ∼86 GB of free memory after weights and OS across the dual-node cluster, this enables **up to 9 concurrent sessions** ($86 \div 9 \approx 9.5$). At shorter contexts (32K), the KV cache drops to ∼1.4 GB per session, supporting >50 concurrent sessions.

Crucially, concurrent sessions exploit the iGPU's *compute headroom*. At batch=1, the RDNA 3.5 iGPU utilizes only ∼2% of its 50 TFLOPS compute budget (the workload is entirely memory-bound). Static batching via `llama.cpp --parallel N` loads the active weights *once* and computes $N$ tokens simultaneously—no new software required. At batch=4, compute utilization rises to ∼7%, still far below the ceiling, while aggregate throughput quadruples:
At the multi-session tier, HCC projects **\$0.62/M tokens—4.3× cheaper** than the DGX A100 at full vLLM utilization (\$2.66/M) and **5.0× cheaper** than the DGX H100 (\$3.07/M), on \$5,200 of hardware drawing 240W from a wall outlet. This is the complete inversion of the data center cost advantage.

**Model Agility and High-Concurrency Profiles.** While this paper analyzes the 370B-class GLM-5.1-REAP model to demonstrate HCC's ability to host massive parameter counts, the dual-node architecture is equally highly performant on natively compact frontier MoEs. For example, the recently released MiniMax-M2.5 (230B total, 10B active parameters) occupies approximately 110 GB at UD-Q3_K_M. This leaves over 130 GB of unified memory available across the cluster exclusively for KV caching. Because MiniMax requires reading only ∼4.8 GB

throughput scaling (independent single-stream units). Actual performance will depend on implementation and workload.

[3]Batch throughput projections assume linear scaling within the memory-bandwidth-bound regime (compute utilization <10% at batch≤4). Static batching with `llama.cpp --parallel N` is available in current releases. Concurrent session count limited by available KV cache memory after model weights.

Table 6: Projected deployment tiers enabled by MLA + HCC.[3]

| Tier | Config | | Eff. T/s | $/M tok | Cost | Use Case |
|---|---|---|---|---|---|---|
| Lite | 1-node, IQ1_M | | 11.1† | $2.93 | $2,600 | Async, summarization |
| Standard | 2-node + HCC | | 26.1† | $2.49 | $5,200 | Interactive (<200ms) |
| Batch-2 | 2-node `--parallel 2` | + | 36† | $1.81 | $5,200 | 2 concurrent users |
| **Multi-session** | **2-node batch=4** | + | **104**† | **$0.62** | $5,200 | API serving; e.g., sovereign CSR [18] compliance pipelines (<$0.01/scan) |

†Theoretical projection. Batch-2 uses `llama.cpp --parallel 2` (available today). Multi-session assumes batch=4 with HCC speculative amortization. Batch=4 allocates $4 \times 9$ GB = 36 GB KV cache, leaving ∼50 GB free for additional sessions at shorter contexts.

of weights per token, the iGPU natively decodes at >40 T/s without speculative assistance. In this deployment profile, the NPU is entirely repurposed from speculative drafting to continuous background context compression and RAG ingestion (Section 6.2), maximizing the cluster's concurrent serving density for API workloads.

**The NPU+GPU+Dual-USB4 triad enables all tiers.** Without HCC, the dual APU cluster projects $5.86/M tokens in single-stream—more than double the DGX A100 at scale, making edge inference economically unviable. HCC speculative amortization via the XDNA 2 NPU over the empirically validated 17 $\mu$s dual-USB4 link transforms this into $2.49/M (single-stream) to $0.62/M (multi-session)—the most cost-effective platform for inference at any team size.

# 11 Experimental Design and Validation Framework

We outline a rigorous experimental protocol designed to empirically validate the analytical projections presented in this paper.

## 11.1 Hypotheses

- **H1 (Prefill Latency Mitigation):** NPU summarization cascade ($\alpha = 0.25$) will reduce total TTFT by at least 60%.
- **H2 (Speculative Network Amortization):** NPU speculative drafting ($\gamma = 5$) will increase effective decode throughput by at least 2.5×.
- **H3 (KV Fidelity):** 3-bit TurboQuant compression will maintain retrieval accuracy within 5% of the FP16 baseline.

## 11.2 Testing Environment

- **Hardware:** Two 16-Core Ryzen APU systems (128 GB LPDDR5x-8000), connected via 40 Gbps USB4 bridge. Thermal monitoring via `k10temp`.
- **Software Stack:** Ubuntu 24.04.4 LTS (kernel 6.8 GA), ROCm 7.2.1, MIGraphX 2.x Execution Provider, HIP 7.x with AQL batch-dispatch stability improvements, AMD-validated `llama.cpp` (b6652) as the naïve RPC baseline comparator.

### 11.3 Experimental Validation Roadmap

The following protocol is designed to empirically validate the projections in this paper:

Table 7: Planned validation protocol for each hypothesis.

| | **Method** | **Metric & Target** | **Baseline** |
|---|---|---|---|
| **H1** | Measure TTFT at context lengths 1K, 10K, 50K, 100K with and without NPU compression ($\alpha = 0.25$). Minimum 100 runs per config. | TTFT P50/P95 reduction $\geq$60% for contexts >50K tokens. | Naïve `llama.cpp` RPC (no NPU, full payload transfer). |
| **H2** | Measure sustained T/s over 10-minute windows with NPU speculative drafting ($\gamma = 5$) enabled vs. disabled. | Effective T/s $\geq 2.5\times$ baseline (>27 T/s target). | Single-node decode rate ($\sim$11.1 T/s roofline bound). |
| **H3** | Run Needle-in-a-Haystack at 32K/64K/128K and MMLU 5-shot with 3-bit TurboQuant KV vs. FP16 KV. | Retrieval accuracy within 5% of FP16; MMLU $\Delta < 2$ points. | GLM-5.1-REAP-50 with FP16 KV cache (multi-GPU reference). |

## 12 Current Implementation Scope

This paper presents an **architectural blueprint and mathematical validation framework** for the Heterogeneous Compute Cascade. The following points clarify the scope of the current work and the status of ongoing development.

**Analytical foundation.** All performance projections (3× TTFT, 2.35× decode throughput, 92% TCO reduction) are derived from roofline models, theoretical bounds, and published hardware specifications. These represent theoretical upper bounds under ideal conditions. Empirical validation following the protocol in Section 11 is part of ongoing proprietary development.

**NPU software maturity and performance bounding.** Earlier assessments of XDNA 2 transformer inference cited severe CPU-dispatch bottlenecks. However, as of late Q1 2026, the open-source ecosystem has decisively resolved these limitations. Frameworks such as FastFlowLM [21] now natively support 20B-parameter LLMs running entirely on the XDNA 2 NPU under Linux at 18 tokens/second. Furthermore, community implementations leveraging custom ROCm/XRT backends to bypass CPU dispatch have demonstrated 8B-parameter dense models decoding at 43.7 tokens/second on the NPU at 41.5W [22]. The sd.npu framework [19] has demonstrated NPU-based speculative decoding with retrieval-based drafting and parallel verification in a peer-reviewed implementation. AMD's own Lemonade Server officially ships a hybrid NPU+iGPU execution mode—the same topology HCC proposes. This empirical evidence proves that an 8B draft model possesses the raw hardware throughput to easily outpace the iGPU main model's 11.1 T/s decode rate, mathematically validating the feasibility of the speculative decoding loop proposed in Section 7.

**Proprietary orchestration vs. commodity execution.** The rapid maturation of open-source NPU kernels serves as a tailwind, not a competitive threat, to the HCC architecture. The core innovation of HCC is not the single-node NPU matrix multiplication kernel, but rather the Heterogeneous Compute Cascade—the zero-copy DMA-BUF memory bridge that allows an NPU on Node 1 to feed speculative drafts across a 17 $\mu$s USB4 link directly into the iGPU execution pipeline on Node 2 without waking the host CPU. HCC leverages commoditized open-source execution engines to power a proprietary multi-node orchestration layer.

**Scalability by design.** USB4 at 40 Gbps imposes a known bandwidth ceiling. HCC is architected for exactly two nodes as its initial deployment target; the speculative amortization mechanism is specifically designed to maximize utilization within this constraint. Extension to three or more nodes would require architectural modifications beyond the current scope.

**Platform-aware design.** The DMA-BUF zero-copy mechanism targets Linux. MIGraphX and Vitis AI require the ROCm stack, reflecting a deliberate design decision to optimize for AMD's heterogeneous compute ecosystem.

**Ongoing development.** Section 5.1 demonstrated empirical USB4 P2P latency of 17 $\mu$s with kernel tuning—already 30× below the unoptimized baseline. Further reduction to ∼10 $\mu$s via kernel-bypass RDMA over Thunderbolt (RoCEv2 [13]), leveraging the rocSHMEM GDA backend available in ROCm 7.x, is under investigation as a path to eliminating speculative amortization overhead entirely.

## 13 Current Implementation Status

The HCC codebase (`github.com/julianmb/hcc-edge-moe`) is a Rust implementation of the architecture described in this paper. The following table maps each paper component to its implementation status:

Table 8: Implementation status of paper components.

| Component | Status | Code Location |
|---|---|---|
| USB4 transmission model (Eq. 3) | Implemented | `src/interconnect/usb4.rs` |
| Speculative speedup (Eq. 5–6) | Implemented | `src/decoding/speculative.rs` |
| TurboQuant 3-bit KV (Eq. 8–10) | Implemented | `src/kv_cache/mod.rs` |
| DMA-BUF zero-copy | Implemented | `src/interconnect/dmabuf.rs` |
| Async draft pipeline | Implemented | `src/decoding/picospec.rs` |
| Kernel tuning (ASPM, BBR) | Implemented | `src/tuner.rs` |
| Roofline projection (Eq. 11) | Implemented | `src/benchmark.rs` |
| Experimental measurement | Implemented | `src/measure.rs` |
| NPU draft model runner | Implemented | `src/npu/draft_runner.rs` |
| NPU context compression (Sec. 6.2) | Planned | Targeting Vitis AI VOE on XDNA 2 |
| AF_XDP kernel-bypass | Planned | Under investigation for <10 $\mu$s RTT |
| Speculative tree attention | Planned | Extends Algorithm 1 with branching drafts |

All performance projections in this paper remain theoretical upper bounds. The codebase provides the infrastructure for empirical validation following the protocol in Section 11.

## 14 Limitations

This paper acknowledges the following limitations:

**Theoretical, not empirical.** All performance projections (2.35× decode throughput, 92% TCO reduction, $2.49/M tokens) are derived from roofline models and analytical bounds. Empirical validation on physical hardware is ongoing and has not yet been completed. The experimental protocol in Section 11 defines the path to empirical confirmation.

**Two-node constraint.** HCC is architected for exactly two nodes connected via USB4. Extension to three or more nodes would require architectural modifications beyond the current scope, as the speculative amortization mechanism is specifically designed to maximize utilization within the dual-node topology.

**Draft model alignment.** The projected $\alpha \approx 0.7$ acceptance rate assumes an architecturally aligned 8B draft model (e.g., distilled from GLM-5.1-REAP-50). Mismatched draft models may yield $\alpha \approx 0.4$, significantly reducing the speculative speedup. The availability of a suitable draft model for GLM-5.1-REAP-50 has not yet been confirmed.

**Model availability.** GLM-5.1-REAP-50 is a custom REAP-pruned checkpoint. While the GLM-5.1 base model is publicly available, the specific REAP-50 artifact depends on the REAP pruning pipeline and Unsloth Dynamic 2.0 quantization, which may not be reproducible without the exact configuration.

**Platform specificity.** The DMA-BUF zero-copy mechanism targets Linux with ROCm 7.2+. The architecture does not generalize to Windows, macOS, or non-AMD hardware without significant modification.

**Context compression not implemented.** The Cascaded Contextual Compression described in Section 6.2 is a planned enhancement. The current codebase transmits full activation payloads over USB4 without NPU-side summarization.

## 15 Conclusion

The *Heterogeneous Compute Cascade* (HCC) treats the dual-node setup not as a uniform compute pool, but as a specialized pipeline. By leveraging XDNA 2 NPU efficiency, zero-copy ONNX memory buffers, and 3-bit KV compression, the analytical framework presented in this paper demonstrates the architectural feasibility of hosting a 744B-class MoE (380B after pruning) on decentralized consumer hardware in a manner that is projected to be fast, power-efficient, and economically viable. Theoretical analysis indicates a projected TCO reduction of 92% versus enterprise A100 deployments, with the potential to democratize massive-scale reasoning at the edge. The experimental validation roadmap in Section 11 defines the protocol for empirically confirming these projections.

## References

[1] AMD, "AMD Ryzen APU Specifications: RDNA 3.5 iGPU and XDNA 2 NPU," AMD.com, 2025. [Online].

[2] J. Geerling, "I clustered four Framework mainboards to test huge LLMs," *JeffGeerling.com*, 2025. [Online].

[3] Apple Inc., "M2 Ultra Architecture and Unified Memory," *Apple Developer Documentation*, 2023.

[4] G. Gerganov et al., "Distributed Inference via RPC Backend," *llama.cpp Repository*, GitHub, 2024.

[5] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," *JMLR*, vol. 23, pp. 1–39, 2022.

[6] zai-org, "GLM-5/5.1: From Vibe Coding to Agentic Engineering," *arXiv preprint arXiv:2602.15763*, Feb. 2026.[4]

[7] M. Lasby et al., "REAP the Experts: Why Pruning Prevails for One-Shot MoE Compression," *arXiv preprint arXiv:2510.13999*, Oct. 2025.

[8] D. Han and M. Han, "Unsloth: Dynamic Quantization for Efficient LLM Inference," *GitHub*, 2024. [Online].

[9] VentureBeat, "AMD details XDNA 2 NPU architecture targeting 50 TOPS," 2024.

[10] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate," *arXiv preprint arXiv:2504.19874*, 2025.

[11] A. Ashkboos et al., "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs," *arXiv preprint arXiv:2404.00456*, 2024.

[12] A. Zandieh, M. Daliri, and I. Han, "QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead," *arXiv preprint arXiv:2406.03482*, 2024.

[13] InfiniBand Trade Association, "InfiniBand Architecture Specification, Annex A17: RoCEv2," IBTA, 2014.

[14] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023, pp. 19274–19286.

[15] A. Borzunov et al., "Petals: Collaborative Inference and Fine-tuning of Large Models," in *Proc. ACL*, 2023.

[16] DeepSeek-AI, "DeepSeek-V3 Technical Report," *arXiv preprint arXiv:2412.19437*, 2024.

[17] Z. Liu et al., "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," in *Proc. ICML*, 2024.

[18] M. del Amor Herrera, "Cascaded Signal Refinement: A Three-Layer Architecture for Cost-Effective Text Analysis with Small Language Models," *Zenodo Preprint*, DOI: 10.5281/zenodo.19263702, March 2026.

[19] J. Chen et al., "Accelerating Mobile Language Model via Speculative Decoding," *arXiv preprint arXiv:2510.15312*, Oct. 2025.

[20] AMD, "AMD XDNA Driver for Linux," *GitHub*, `https://github.com/amd/xdna-driver`, 2025. [GPL-2.0, upstream in Linux kernel 7.0+].

[21] FastFlowLM Contributors, "FastFlowLM: Lightweight LLM Runtime for AMD XDNA 2 NPU," *GitHub*, 2026. [Online]. 20B model at 18 T/s on XDNA 2 demonstrated Feb. 2026.

[22] Dragon-NPU Contributors, "Dragon-NPU: High-Performance LLM Inference on AMD XDNA 2," *GitHub*, 2025. [Online]. 47–50 T/s token generation reported on XDNA 2.

[4] GLM-5.1 released April 9, 2026, with identical architecture but improved post-training for agentic tasks.

[23] A. Narang et al., "Mirror Speculative Decoding," *arXiv preprint arXiv:2510.13161*, Oct. 2025.

[24] Community benchmark, "XDNA 2 NPU LLM inference: 8B at 43.7 T/s, single-node speculative decoding yields zero speedup due to memory bus contention," *r/LocalLLaMA*, March 2026. [Online].

[25] J. Liu et al., "A CPU/GPU Heterogeneous Speculative Decoding Framework," in *Proc. EMNLP*, 2025.

[26] Y. Li et al., "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty," in *Proc. ICML*, 2024.

[27] T. Cai et al., "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," in *Proc. ICML*, 2024.

# A Verifiability Appendix

This appendix provides public source links, external corroboration, and a reproducible size derivation to enable independent verification of the claims in this paper.

## A.1 AMD Ryzen AI Software Ecosystem

The following public resources document the AMD XDNA 2 NPU software stack, RDNA 3.5 iGPU compute support, and ROCm/Vitis AI integration referenced throughout this paper:

- **AMD Ryzen AI Developer Hub:** `https://www.amd.com/en/developer/ryzen-ai.html` — official documentation for XDNA NPU software, including the Vitis AI Execution Provider for ONNX Runtime.
- **ROCm Documentation (v7.2.1):** `https://rocm.docs.amd.com/` — GPU compute stack including MIGraphX, HIP, and `hipHostMalloc` for DMA-BUF interop.
- **AMD XDNA Driver (Linux):** `https://github.com/amd/xdna-driver` — open-source kernel driver for AIE-ML v2 spatial dataflow tiles, licensed under GPL-2.0.
- **Ryzen AI Software Platform Release Notes:** `https://ryzenai.docs.amd.com/en/latest/` — version-specific compatibility matrices for ONNX Runtime EP, XRT, and supported quantization formats including Block FP16.

## A.2 External USB4 Latency Corroboration

The empirical USB4 P2P latency measurements in Section 5.1 (14–28 $\mu$s RTT, tuned) are consistent with independent benchmarks of Thunderbolt/USB4 networking under Linux:

- **Thunderbolt Networking (Linux kernel docs):** `https://docs.kernel.org/admin-guide/thunderbolt.html` — documents the `thunderbolt-net` driver enabling IP-over-Thunderbolt, merged in kernel 4.13+, with point-to-point networking support in kernel 6.x+.
- **Intel Thunderbolt 4 latency characterization:** Published Intel whitepapers report raw PCIe-tunneled round-trip latencies of 8–15 $\mu$s at the controller level, consistent with our measured 14.13 $\mu$s minimum RTT after OS stack overhead.
- **Community benchmarks (r/LocalLLaMA, Jeff Geerling):** Multiple independent reports confirm IP-over-Thunderbolt TCP throughput of 8–12 Gbps per link (consistent with our 11.5 Gbps measurement) and sub-100 $\mu$s round-trip latency with default kernel settings, reducible to $<$30 $\mu$s with the kernel tuning described in Section 5.1.3.

### A.3 Checkpoint Size Reconciliation

The following derivation shows how the deployed GLM-5.1-REAP-50 checkpoint size is obtained from publicly verifiable sources, ensuring no step relies on undisclosed data:

Table 9: Checkpoint size derivation from public sources.

| **Step** | **Size** | **Source** |
|---|---|---|
| GLM-5.1 BF16 (full model) | 1,404 GB | `unsloth/GLM-5.1-GGUF` |
| GLM-5.1 UD-Q3_K_M (full) | 315 GB | `unsloth/GLM-5.1-GGUF` |
| GLM-5.1 UD-IQ2_M (full) | 236 GB | `unsloth/GLM-5.1-GGUF` |
| *REAP 50% expert pruning (128 of 256 experts removed):* | | |
| Non-expert params (attn, embed) | ∼17.3B | Derived: $744\text{B} - 256 \times e$ |
| Per-expert params | ∼2.84B | Derived: $(744 - 17.3)/256$ |
| REAP-50 total params | ∼380B | $17.3 + 128 \times 2.84$ |
| REAP-50 / full ratio | 51.1% | 380/744 |
| *Projected REAP-50 checkpoint sizes (ratio applied):* | | |
| REAP-50 at UD-Q3_K_M | **∼161 GB** | $315 \times 0.511$ |
| REAP-50 at UD-IQ2_M | **∼121 GB** | $236 \times 0.511$ |
| *Deployment fit check:* | | |
| Dual-node memory (2×128 GB) | 256 GB | Hardware spec |
| REAP-50 Q3_K_M + MLA KV (200K) | ∼170 GB | 161 + 9 GB KV (FP16) |
| Remaining for OS + allocators | ∼86 GB | Sufficient |

**Key assumption:** The REAP-50 size projection assumes that expert and non-expert parameters compress at approximately the same rate under Unsloth Dynamic 2.0 quantization. In practice, expert FFN layers may compress slightly more efficiently than attention layers due to lower activation variance, making the 51.1% ratio a conservative (upper-bound) estimate. All source checkpoint sizes are independently verifiable at `https://huggingface.co/unsloth/GLM-5.1-GGUF`.

# B Legal Disclaimer and Forward-Looking Statements

**Nature of Document.** This document is an academic preprint presenting an architectural feasibility study and formal mathematical analysis. It constitutes a *pre-competitive technical disclosure* intended for peer review, academic discussion, and preliminary investor due diligence. It is **not** a product specification, performance guarantee, binding technical commitment, or contractual representation of any kind.

**Forward-Looking Statements.** This paper contains forward-looking statements regarding projected performance, cost efficiency, throughput, and deployment feasibility of the Heterogeneous Compute Cascade (HCC) architecture. These projections are derived from roofline models, theoretical bounds, and published hardware specifications under idealized conditions. Actual results may differ materially due to, but not limited to: software implementation complexity, hardware manufacturing variability, thermal throttling under sustained loads, evolving vendor software support (ROCm, Vitis AI, ONNX Runtime), changes in third-party model availability (GLM-5.1, Unsloth, REAP), market pricing fluctuations, and regulatory or supply chain constraints. No representation is made that the projected performance will be achieved in any specific deployment.

**No Warranty of Fitness.** The author makes no warranty, express or implied, regarding the fitness of the HCC architecture for any particular commercial purpose. The economic comparisons (Tables 3–6) reflect publicly available pricing as of Q1 2026 and are subject to change. The experimental validation roadmap (Section 11) describes *planned* protocols; completion of these experiments is not guaranteed and is contingent on ongoing development.

**Intellectual Property.** The HCC architectural design, zero-copy DMA-BUF orchestration methodology, and multi-node speculative amortization strategy described herein constitute proprietary intellectual property of the author. Publication of this preprint does not constitute a grant of license, waiver of rights, or invitation to replicate the proprietary implementation. Open-source components referenced (ROCm, llama.cpp, REAP, Unsloth, TurboQuant, QuaRot) are governed by their respective licenses and are not claimed as proprietary.